

TELE-SEVERINO

PROJETO INTEGRADOR – BLOCO 3: ARQUITETURA DO SISTEMA

O Tele Severino é uma plataforma digital criada para conectar, de forma rápida e acessível, pessoas que possuem dúvidas técnicas ou práticas a profissionais experientes de diversas áreas, por meio de atendimento remoto via vídeo ou áudio, com cobrança por minuto de uso.

Este projeto foi desenvolvido como parte da terceira entrega oficial do Projeto Integrador, correspondente ao Bloco 3 – Arquitetura do Sistema.

INTEGRANTES DO GRUPO 13

Data de realização: 25/03/2026

Luciano S. da Silva Junior—RA: 926114218—lucianososiljunior@uni9.edu.br

Israel de Albuquerque — RA: 926116180 — israelalbuquerque@uni9.edu.br

Edvanio Maximo da Silva — RA: 926114228 — silva.edvanio@uni9.edu.br

Roger Sousa Funaki — RA: 926115899 — roger.funaki@uni9.edu.br

Cicero Silva de Almeida Junior — RA: 1726104302 — cj.almeida@uni9.edu.br

DIAGRAMA C4 — NÍVEL CONTEXT (C1)

O Diagrama de Contexto mostra até onde o Tele Severino atua e com quem ele se comunica. Nele, a plataforma é apresentada como o sistema central, conectando clientes, estabelecimentos parceiros e serviços externos, como APIs de pagamento e localização. O foco desse diagrama é deixar claro quais atores interagem com o sistema e quais informações entram e saem da plataforma, sem detalhar programação, banco de dados ou estrutura interna.

Relatório Técnico do Contexto:

Atores do Sistema:

- **Cliente:** Usuário final que acessa a plataforma em busca de um atendimento remoto especializado (áudio/vídeo). É responsável por iniciar a chamada e realizar o pagamento;
- **Profissional:** Especialista técnico cadastrado que disponibiliza seu tempo na plataforma para realizar os atendimentos e recebe os repasses financeiros proporcionalmente aos minutos trabalhados;
- **Administrador:** Usuário interno que monitora a saúde da plataforma, extrai relatórios gerenciais e resolve disputas ou suporte;
- **Sistema Central (Tele Severino):** A plataforma central que gerencia as conexões, calcula a tarifação do atendimento por minuto, controla os perfis de usuários e

orquestra as regras gerais do negócio.

Sistemas Externos Integrados:

- **API WhatsApp (Meta):** Dispara notificações automáticas de confirmação de conta, lembretes de chamadas e alertas operacionais;
- **API Mapas (Google Maps):** Utilizada para fins de validação geográfica ou localização regional dos profissionais.
- **Gateway de Pagamento (Mercado Pago):** Instituição financeira responsável por processar as cobranças automáticas via Pix ou Cartão de Crédito e validar o status do pagamento.

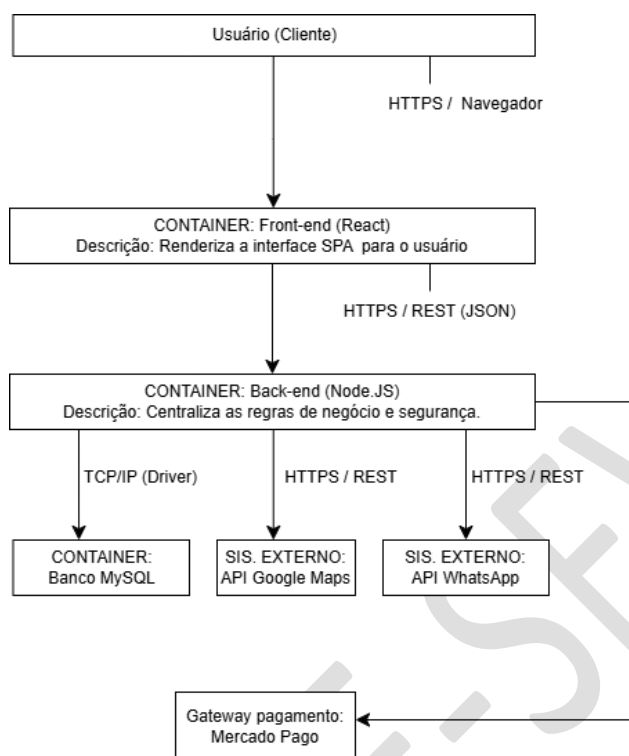


Figura 1- Diagrama de Contexto (C1) do Sistema Tele Severino. Código-fonte do Diagrama

DIAGRAMA C4 — NÍVEL CONTAINER (C2)

O Diagrama de Containers mostra como o Tele Severino foi dividido internamente para funcionar na prática. Nele, o sistema é separado em partes como front-end, back-end e banco de dados, deixando mais claro o papel de cada componente e a forma como eles se comunicam. Esse diagrama ajuda a entender a estrutura da aplicação e as tecnologias usadas em cada camada, sem entrar no nível de código.

Relatório Técnico e Justificativas (C2):

- **Front-end (React):** Escolhido para a criação de uma Single Page Application (SPA) dinâmica e de alta performance. O React permite a componentização da interface e uma renderização ágil em tempo real, garantindo uma experiência fluida para clientes e profissionais em dispositivos móveis e web;

- **Back-end (Node.js):** Selecionado devido à sua arquitetura orientada a eventos e I/O não-bloqueante. O Node.js lida de forma extremamente eficiente com requisições assíncronas simultâneas, característica vital para o controle de concorrência e cronometragem de chamadas faturadas por minuto;
- **Banco de Dados (MySQL):** Banco de dados relacional (SGBD) adotado para garantir a consistência rigorosa das informações (propriedades ACID). É fundamental para assegurar a integridade dos históricos de ligações, saldos financeiros e dados cadastrais sensíveis;
- **Segurança e Protocolos de Rede:** A arquitetura foi desenhada seguindo as melhores práticas de segurança de infraestrutura. O contêiner de Front-end comunica-se exclusivamente com o Back-end através do protocolo HTTPS / REST (JSON). O Back-end em Node.js isola e protege o banco de dados MySQL do acesso externo direto, comunicando-se com ele em rede interna via Driver SQL/TCP. Além disso, o Back-end atua como uma barreira segura para consumir as APIs externas de pagamento, mapas e mensageria, prevenindo a exposição de chaves privadas no cliente.

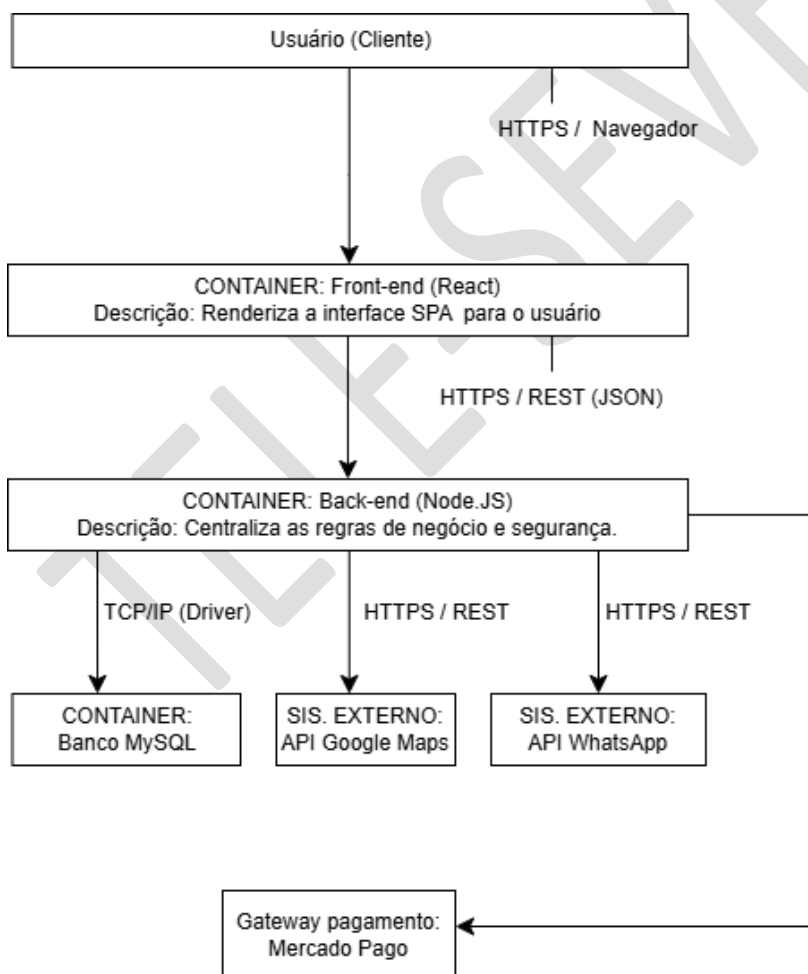


Figura 2- Diagrama de Containers (C2) detalhando tecnologias.

NÍVEL 3: DIAGRAMA DE COMPONENTES (C3)

O Diagrama de Componentes mostra como o back-end do Tele Severino foi organizado por dentro. Ele separa o sistema em módulos menores, como rotas, controllers, serviços e acesso ao banco de dados, deixando mais claro o papel de cada parte no processamento das requisições. A ideia é mostrar como o código se conecta internamente para fazer o sistema funcionar de forma organizada.

Relatório Técnico dos Componentes (C3):

- **Routes & Controllers:** Camada de interface de rede do Back-end. É responsável por expor os endpoints públicos da API, interceptar as chamadas HTTP enviadas pelo React, efetuar a validação prévia dos dados de entrada (sanitização com Zod/Joi) e despachar a requisição para a camada correspondente;
- **Business Services:** O núcleo lógico do sistema. Centraliza todas as regras de negócio do Tele Severino, tais como: verificação de disponibilidade do profissional, cálculo do valor final do atendimento baseado nos minutos cronometrados e aplicação de regras de cupons ou taxas da plataforma;
- **Data Models / Repositories:** Camada de persistência que abstrai o banco de dados utilizando uma ORM (como Prisma ou Sequelize). Transforma as instruções Javascript do Node.js em comandos SQL válidos para o MySQL de forma automatizada, blindando a aplicação contra ataques de SQL Injection;
- **Integration Services / Gateways:** Módulo isolado especializado na comunicação de saída do servidor. Ele empacota os dados e gerencia os tokens de autenticação necessários para consumir os endpoints do Mercado Pago, WhatsApp e Google Maps, garantindo que o restante do Back-end não precise conhecer os detalhes internos dessas APIs terceiras.

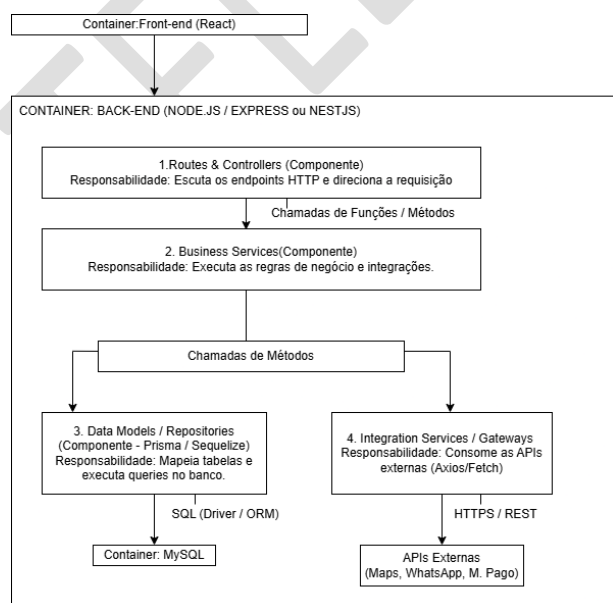


Figura 3- Diagrama de Componentes (C3) do Back-end Node.js.Código-fonte do Diagrama

MODELAGEM DE DADOS — SQL (MySQL)

Diagrama Entidade-Relacionamento (DER) ou Modelo Relacional

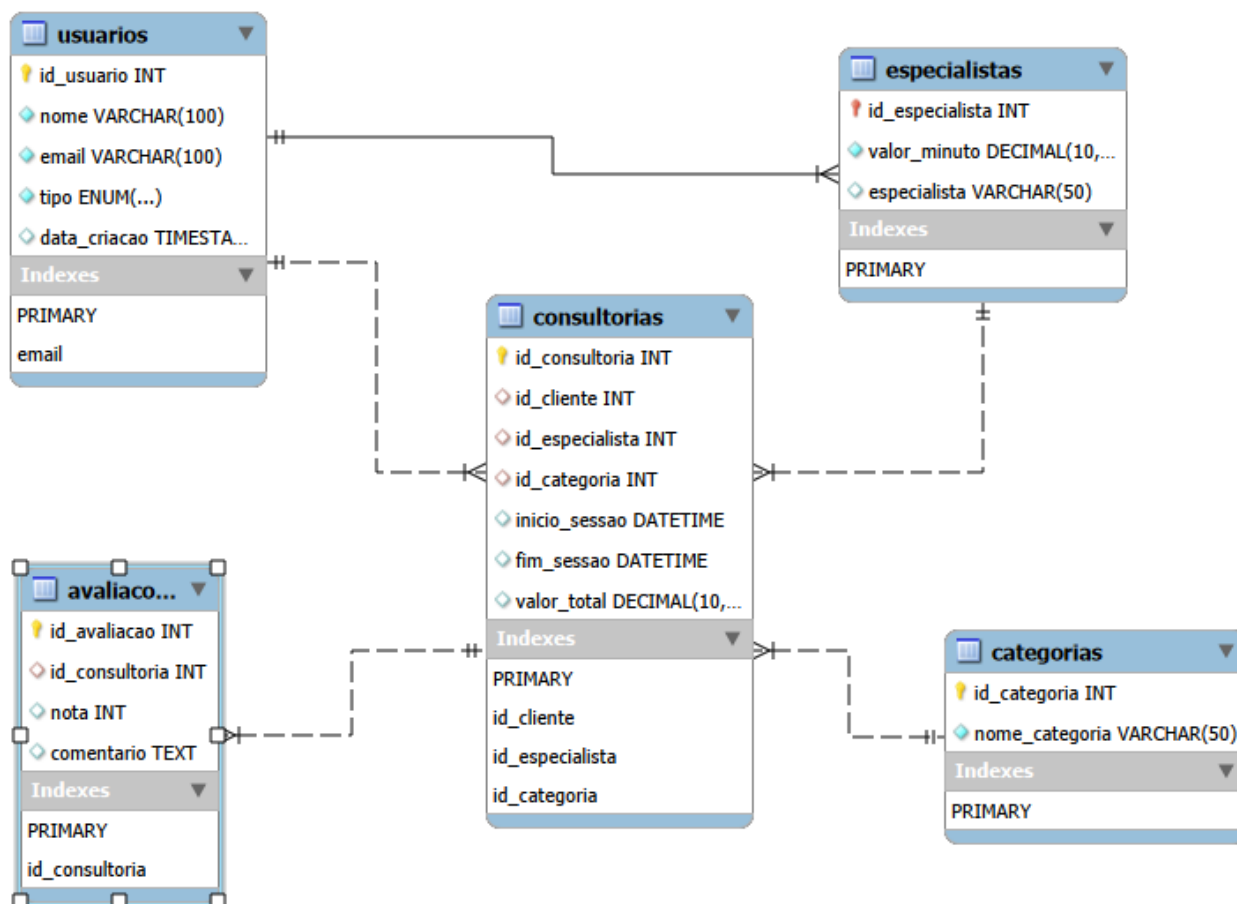


Figura 4- Diagrama Banco de Dados

Script SQL

-- CRIAÇÃO DO BANCO

```
CREATE DATABASE tele_severino;
```

```
USE tele_severino;
```

-- 1. USUÁRIOS

```
CREATE TABLE usuarios (
```

```
  id_usuario INT PRIMARY KEY AUTO_INCREMENT,
```

```
  nome VARCHAR(100) NOT NULL,
```

```
  email VARCHAR(100) UNIQUE NOT NULL,
```

```
  senha VARCHAR(255) NOT NULL,
```

```
  tipo ENUM('CLIENTE', 'ESPECIALISTA') NOT NULL,
```

```
  data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```



-- 2. ESPECIALISTAS

```
CREATE TABLE especialistas (  
    id_especialista INT PRIMARY KEY,  
    especialidade VARCHAR(50) NOT NULL,  
    valor_minuto DECIMAL(10,2) NOT NULL,  
  
    FOREIGN KEY (id_especialista)  
        REFERENCES usuarios(id_usuario)  
);
```

-- 3. CATEGORIAS

```
CREATE TABLE categorias (  
    id_categoria INT PRIMARY KEY AUTO_INCREMENT,  
    nome_categoria VARCHAR(50) UNIQUE NOT NULL  
);
```

-- 4. CONSULTORIAS

```
CREATE TABLE consultorias (  
    id_consultoria INT PRIMARY KEY AUTO_INCREMENT,  
  
    id_cliente INT NOT NULL,  
    id_especialista INT NOT NULL,  
    id_categoria INT NOT NULL,  
  
    inicio_sessao DATETIME,  
    fim_sessao DATETIME,  
  
    valor_total DECIMAL(10,2),  
  
    FOREIGN KEY (id_cliente)  
        REFERENCES usuarios(id_usuario),  
  
    FOREIGN KEY (id_especialista)  
        REFERENCES especialistas(id_especialista),  
  
    FOREIGN KEY (id_categoria)
```





```
REFERENCES categorias(id_categoria)
);
```

-- 5. AVALIAÇÕES

```
CREATE TABLE avaliacoes (
    id_avaliacao INT PRIMARY KEY AUTO_INCREMENT,

    id_consultoria INT NOT NULL,

    nota INT CHECK (nota BETWEEN 1 AND 5),
    comentario TEXT,

    FOREIGN KEY (id_consultoria)
        REFERENCES consultorias(id_consultoria)
);
```

MODELAGEM COLEÇÃO MONGODB

```
"database": "tele_severino",
"tables": [
    {
        "name": "usuarios",
        "columns": [
            {
                "name": "id_usuario",
                "type": "INT",
                "primary_key": true,
                "auto_increment": true
            },
            {
                "name": "nome",
                "type": "VARCHAR(100)",
                "nullable": false
            },
            {
                "name": "email",
```





```
"type": "VARCHAR(100)",
"unique": true,
"nullable": false
},
{
  "name": "tipo",
  "type": "ENUM('CLIENTE', 'ESPECIALISTA')",
  "nullable": false
},
{
  "name": "data_criacao",
  "type": "TIMESTAMP",
  "default": "CURRENT_TIMESTAMP"
}
]
},
{
  "name": "especialistas",
  "columns": [
    {
      "name": "id_especialista",
      "type": "INT",
      "primary_key": true
    },
    {
      "name": "valor_minuto",
      "type": "DECIMAL(10,2)",
      "nullable": false
    },
    {
      "name": "especialista",
      "type": "VARCHAR(50)"
    }
  ],
  "foreign_keys": [
    {
```





```
        "column": "id_especialista",
        "references": "usuarios(id_usuario)"
    }
]
},
{
    "name": "categorias",
    "columns": [
        {
            "name": "id_categoria",
            "type": "INT",
            "primary_key": true,
            "auto_increment": true
        },
        {
            "name": "nome_categoria",
            "type": "VARCHAR(50)",
            "nullable": false
        }
    ]
},
{
    "name": "consultorias",
    "columns": [
        {
            "name": "id_consultoria",
            "type": "INT",
            "primary_key": true,
            "auto_increment": true
        },
        {
            "name": "id_cliente",
            "type": "INT"
        },
        {
            "name": "id_especialista",
```





```
    "type": "INT"
  },
  {
    "name": "id_categoria",
    "type": "INT"
  },
  {
    "name": "inicio_sessao",
    "type": "DATETIME"
  },
  {
    "name": "fim_sessao",
    "type": "DATETIME"
  },
  {
    "name": "valor_total",
    "type": "DECIMAL(10,2)"
  }
],
"foreign_keys": [
  {
    "column": "id_cliente",
    "references": "usuarios(id_usuario)"
  },
  {
    "column": "id_especialista",
    "references": "especialistas(id_especialista)"
  },
  {
    "column": "id_categoria",
    "references": "categorias(id_categoria)"
  }
]
},
{
  "name": "avaliacoes",
```





```
"columns": [  
  {  
    "name": "id_avaliacao",  
    "type": "INT",  
    "primary_key": true,  
    "auto_increment": true  
  },  
  {  
    "name": "id_consultoria",  
    "type": "INT"  
  },  
  {  
    "name": "nota",  
    "type": "INT",  
    "check": "nota BETWEEN 1 AND 5"  
  },  
  {  
    "name": "comentario",  
    "type": "TEXT"  
  }  
],  
"foreign_keys": [  
  {  
    "column": "id_consultoria",  
    "references": "consultorias(id_consultoria)"  
  }  
]  
},  
{  
  "name": "produtos",  
  "columns": [  
    {  
      "name": "id",  
      "type": "INT",  
      "primary_key": true,  
      "auto_increment": true
```





```
{
  },
  {
    "name": "nome",
    "type": "VARCHAR(100)",
    "nullable": false
  },
  {
    "name": "detalhes",
    "type": "JSON",
    "nullable": false
  }
]
}
}
```

MODELAGEM DE ESTRUTURA REDIS (CHAVE-VALOR, HASH OU SORTED SET)

Chave: sessao:ativa:482

Campos (Fields) e Valores:

- └─ id_cliente: "12"
- └─ id_especialista: "5"
- └─ valor_minuto: "3.50"
- └─ inicio_sessao: "1717621200" (Timestamp Unix)
- └─ acumulado_atual: "10.50"

Justificativa do Caso

UsoPerformance em Tempo Real: Durante a chamada, o aplicativo cliente precisa atualizar uma interface mostrando: "Você já está em chamada há 3 minutos e o custo atual é R\$ 10,50". O Redis processa leituras e atualizações em tempo de microssegundos ($O(1)$), poupando seu banco relacional de ler a tabela consultorias repetidamente;

Agilidade no Encerramento: Assim que o cliente ou o especialista desliga, a aplicação executa um comando rápido HGETALL sessao:ativa:482, calcula a diferença exata de tempo entre o fim_sessao e o inicio_sessao, multiplica pelo valor_minuto e faz um único INSERT/UPDATE definitivo na sua tabela consultorias do MySQL;

Resiliência a Quedas: Se a internet de um dos usuários cair no meio da sessão, o estado





atualizado do valor cobrado até aquele exato segundo está salvo de forma segura na memória do Redis, evitando prejuízos financeiros para ambas as partes.

Repositório *GitHub*

Link do repositório: <https://github.com/CiriloDevelope/Tele-Severino>

URL SSH: <git@github.com:CiriloDevelope/Tele-Severino.git>

INSTRUÇÕES DE USO

Clonar o repositório

- *git clone* git clone <git@github.com:CiriloDevelope/Tele-Severino.git>

